

SoDam-Reverse-Eng 완전 사용 설명서 (한국어)

컴퓨터나 AI를 처음 써보는 분도 이 가이드 하나로 끝냅니다.

처음부터 끝까지 따라가면 됩니다. 중간에 막히면 → [11장 FAQ](#) 또는 [10장 문제 해결](#)

목차

1. [이 도구가 뭔가요? \(쉬운 설명\)](#)
2. [설치 방법 \(처음 한 번\)](#)
3. [SoDam 형제 플러그인과 함께 쓰기](#)
4. [명령어 상세 안내](#)
5. [첫 분석 시작하기 \(단계별 따라하기\)](#)
6. [보안·데이터 흐름 상세](#)
7. [아키텍처 \(내부 구조\) 상세](#)
8. [파일·문서 위치 가이드](#)
9. [고급 기능 안내](#)
10. [문제 해결·오류 대처](#)
11. [FAQ 자주 묻는 질문 25가지](#)
12. [라이선스·저작권·상업적 용도 상세](#)

1. 이 도구가 뭔가요? (쉬운 설명)

“쉬운 말로만” 설명

여러분: "이 코드가 뭐 하는 거야?"

AI: "이건 로그인 기능을 처리하는 코드예요.
사용자가 아이디와 비밀번호를 입력하면
데이터베이스에서 확인하고, 맞으면 열어주는 구조예요."

그게 전부입니다.

코드를 AI에게 보여주면, AI가 쉬운 한국어로 설명해 줍니다. 분석 결과는 깔끔한 보고서로 정리됩니다.

왜 만들었나요?

- 예전에 만든 코드가 어떻게 작동했는지 기억이 안 날 때
- 다른 개발자가 쓴 코드(허가받은 것)를 이해해야 할 때
- “이 부분에 버그가 있나?” 궁금할 때
- 보안 취약점이 어디 있는지 찾고 싶을 때

절대로 안 되는 것 (명확하게 고지합니다)

크랙, DRM 우회, 인증 우회, 비밀번호 추출, 라이선스 우회

이런 요청은 거절합니다. 거절 이유를 설명하고 분석을 중단합니다. 아무리 요청해도 방법을 알려주지 않습니다. 이것은 이 도구의 가장 중요한 기능입니다.

2. 설치 방법 (처음 한 번)

한 번만 하면 됩니다. 이후에는 켜기만 하면 됩니다.

2-1. 사전 준비물 확인

준비물 1: Node.js (버전 18 이상)

이미 있는지 확인하는 방법:

1. 키보드에서 **Windows 키** 누르기
2. 검색창에 powershell 입력 → **Windows PowerShell** 클릭
3. 검은 창이 열리면 아래를 입력하고 **Enter**:

```
node --version
```

4. 아래처럼 숫자가 나오면 OK:

v20.19.0 ← 이런 형태면 됩니다 (v18 이상)

5. 숫자가 안 나오거나 v18보다 낮으면 → 아래 “설치 방법” 따라하기

Node.js 설치 방법 (없는 경우):

1. 웹브라우저(크롬, 엣지, 파이어폭스 등) 열기
2. 주소창에 `nodejs.org` 입력 → Enter
3. 초록색 “**LTS**” 버튼 클릭 (안정 버전)
4. `.msi` 파일이 내려받아지면 더블클릭
5. “**다음(Next)**” 버튼을 계속 누르고 “**설치(Install)**” 클릭
6. 설치가 끝나면 **컴퓨터 재부팅** (재부팅 안 하면 인식 안 될 수 있음)
7. 재부팅 후 PowerShell 다시 열어서 `node --version` 확인

💡 **팁:** “컴퓨터 재부팅”은 윈도우 시작 버튼 → 전원 → 다시 시작

준비물 2: Claude Code

이 가이드를 Claude Code에서 보고 있다면 이미 있는 것입니다. 없다면 Anthropic 공식 사이트에서 “Claude Code”를 검색해서 설치하세요.

2-2. 플러그인 설치 (5단계)

⚠️ 시작 전 꼭 읽기 — 실패 원인 1위

Claude Code는 **컬** 때의 폴더를 기준으로 명령어를 읽습니다.

- ❌ 틀린 방법: `C:\Users\이름` 홈 폴더에서 `claude` 실행
- ✅ 맞는 방법: 내 프로젝트 폴더(`D:\내프로젝트` 등)에서 `claude` 실행

채팅창에서 `cd` 다른폴더를 입력해도 명령어는 다시 읽히지 않습니다. **반드시 프로젝트 폴더에서 Claude Code를 시작하세요.**

단계 1: 올바른 폴더에서 Claude Code 시작

PowerShell을 열고:

```
cd D:\내프로젝트폴더
claude
```

💡 `D:\내프로젝트폴더` 부분을 실제 내 프로젝트 경로로 바꾸세요. 프로젝트가 없어도 됩니다. 분석하고 싶은 코드가 있는 폴더면 됩니다. 예: `cd D:\내코드\login-app`

단계 2: 마켓플레이스 등록

Claude Code 채팅창에 아래를 입력:

 이 플러그인은 비공개 저장소입니다. 받은 방식에 따라 아래 중 하나를 선택하세요.

방법 A — GitHub 저장소에서 복제 (저장소 초대를 받은 분)

 GitHub 계정 + 저장소 초대 필요. 초대 요청: startmxk@gmail.com

1. PowerShell에서 복제:

```
git clone https://github.com/sodam-ai/SoDam-Reverse-Eng.git
```

2. 복제된 폴더 경로 확인 (예: C:\Users\내이름\SoDam-Reverse-Eng)

3. Claude Code 채팅창에 입력:

```
/plugin marketplace add C:\Users\내이름\SoDam-Reverse-Eng
```

 내이름을 실제 윈도우 사용자 이름으로 바꾸세요.

방법 B — 폴더/압축 파일로 전달받은 분 (git 불필요)

1. 받은 파일을 원하는 위치에 압축 해제 (예: C:\Users\내이름\SoDam-Reverse-Eng)

2. Claude Code 채팅창에 실제 경로 입력:

```
/plugin marketplace add C:\Users\내이름\SoDam-Reverse-Eng
```

 경로에 공백이 있으면: `/plugin marketplace add "C:\내 폴더\SoDam-Reverse-Eng"`

단계 3: 플러그인 설치

마켓플레이스 등록 후 바로 입력:

```
/plugin install sodam-reverse@sodamreverse-marketplace
```

또는 /plugin 입력 → 메뉴에서: 1. **Browse marketplaces** 선택 2. **sodamreverse-marketplace** 선택 3. **sodam-reverse** 선택 4. **Install** 클릭

단계 4: 완전 재시작

 이 단계를 빠뜨리면 명령어가 안 됩니다. 반드시!

플러그인은 **Claude Code 시작 시에만** 로드됩니다.

1. 채팅창에 `/exit` 입력 → Enter
2. PowerShell 창 **완전히 닫기** (X 버튼)
3. PowerShell **새로 열기**
4. 반드시 **프로젝트 폴더**에서 시작:

```
cd D:\내프로젝트폴더
claude
```

단계 5: 설치 확인 (진단 명령어)

Claude Code가 열리면 아래를 입력:

```
/re-ping
```

성공 응답:

Pong! /re-ping 정상 작동합니다.

이 응답이 오면 플러그인이 제대로 로딩됐습니다.

2-3. 안전장치 검증 (3층 점검)

```
/re-selftest
```

기대 결과:

[1층] AI 거부 규칙	✅	정상
[2층] deny-hook	✅	정상
[3층] 무결성 점검	✅	정상

3개 모두 **✅**여야 합니다. 일부가 **❌**이면 → [10장 문제해결](#)

2-4. 무결성 해시 등록 (3층 완전 활성화)

3층 무결성 점검이 제대로 작동하려면 해시를 등록해야 합니다.

1. `/re-selftest` 실행
2. 출력 중 SHA-256 해시: 뒤의 64자리 문자열 복사
3. 플러그인 폴더 → `references/integrity.json` 열기
4. "hooks/re-deny-guard.mjs" 항목 값에 복사한 해시 붙여넣기

- 5. 파일 저장
- 6. 다시 `/re-selftest` 실행 → 3층  확인

2-5. 형제 시너지 설정 (선택 사항)

SoDam-Harness가 설치된 경우 아래를 실행하세요:

```
node scripts/re-inject-harness.mjs
```

“RE 규칙 N개 주입 완료” 메시지가 나오면 성공.

3. SoDam 형제 플러그인과 함께 쓰기

6형제 개요

SoDam은 6개의 플러그인이 하나의 팀처럼 작동합니다:

번호	이름	역할
1	SoDam-Harness	전체 안전벨트 (백업·규칙 중앙관리)
2	SoDam-Loop	반복 작업 안전 제어
3	SoDam-Context	CLAUDE.md 건강검진
4	SoDam-Agentic	계획·보고서 재검수
5	SoDam-Prompt	자연어 요청 품질 향상
6	SoDam-Reverse	코드 분석 (이 플러그인)

권장 설치 순서: 1번부터 6번까지 순서대로

각 형제와 Reverse의 관계

Harness와 함께 쓸 때

Harness는 Reverse의 “안전 규칙 선생님” 역할입니다. 설치하면 Reverse의 위험 패턴 차단 규칙이 Harness에 공유됩니다.

연결 방법:

```
node scripts/re-inject-harness.mjs
```

효과: hook 하나가 두 플러그인의 규칙을 모두 담당합니다 (중복 없음).

중요: Harness·Loop가 설치된 경우, c:\Users\이름 홈 폴더에서 Claude Code를 시작하면 정상 작업도 막힐 수 있습니다. **반드시 프로젝트 폴더에서** 시작하세요.

Context와 함께 쓸 때

Context는 CLAUDE.md 파일이 올바른지 정기 점검해 주는 도구입니다. RE 범위(방어·교육·본인소유물 전용) 누락을 자동으로 감지합니다.

연결 방법:

```
node scripts/re-inject-context.mjs
```

명령어 출력이 나오면 그대로 복붙하여 실행.

Agentic과 함께 쓸 때

Agentic은 보고서를 비개발자 눈높이로 재검수해 주는 도구입니다.

사용 방법: /re-start 실행 후 AI에게 말하기:

"이 보고서를 초보자도 이해할 수 있게 다시 설명해줘"

그러면 Agentic의 easy-reviewer가 자동으로 개입합니다.

형제 전체 상태 확인

```
node scripts/check-family.mjs
```

출력 예시:

SoDam 6형제 상태 점검

Harness	✅	설치됨
Loop	✅	설치됨
Context	❌	미설치
Agentic	✅	설치됨
Prompt	❌	미설치
Reverse	✅	(현재 플러그인)

활성 시너지:

[Harness+Reverse]	RE 규칙 공유	✅
[Agentic+Reverse]	재검수 트리거	✅
[Context+Reverse]	스코프 점검	❌ (미연결)

4. 명령어 상세 안내

/re-ping — 설치 확인

언제: 설치 직후, 또는 “명령어가 잘 되나?” 확인할 때

입력:

```
/re-ping
```

정상 출력:

Pong! /re-ping 정상 작동합니다.

이상 출력 (또는 아무 반응 없음): → 플러그인이 로드 안 됐습니다. [10장 Q1](#) 참고.

/re-start — 분석 시작

언제: 새로운 코드를 분석하고 싶을 때

입력 형식:

```
/re-start [파일경로]
```

예시:

```
/re-start src/login.js  
/re-start C:\내프로젝트\auth\user.py  
/re-start app.js
```

분석 흐름: 1. AI가 소유권을 확인합니다 (“이 코드가 본인 것인가요?”) 2. AI가 책임 동의를 받습니다 (“방어 목적에 동의하시나요?”) 3. 분석이 시작됩니다 4. 표준 보고서가 출력됩니다

동의 방법:

예 ← 이렇게 입력하세요

“아마도요”, “그런 것 같아요” 같은 애매한 답변은 동의로 처리되지 않습니다.

/re-report — 보고서 재출력

언제: 이전 분석 보고서를 다시 보고 싶을 때

입력:


```
/re-report
```

출력: 가장 최근 분석 결과의 전체 보고서

/re-selftest — 안전장치 3층 점검

언제: 설치 직후, 또는 “안전장치가 잘 작동하나?” 확인할 때

입력:

```
/re-selftest
```

정상 출력:

- [1층] AI 거부 규칙

[2층] deny-hook

[3층] 무결성 점검
-
- 정상

정상

정상

이상 출력 (일부 ✖): → [10장 Q6](#) 참고

앞으로 추가될 명령어

명령어	추가 시기	설명
/re-android	Phase 2 예정	Android APK 파일 분석
/re-binary	Phase 3 예정	실행 파일(.exe 등) 분석

5. 첫 분석 시작하기 (단계별 따라하기)

분석 전 체크리스트

- ☐ Node.js v18 이상 설치됨
- ☐ Claude Code가 **프로젝트 폴더**에서 실행 중
- ☐ /re-ping → “Pong!” 응답 확인
- ☐ /re-selftest → 3개 확인
- ☐ 분석할 파일이 내 소유이거나 분석 허가를 받은 것임

실제 분석 따라하기

1단계: Claude Code 채팅창에 입력

```
/re-start src/login.js
```

(예시입니다. 실제 내 파일 경로를 입력하세요)

2단계: 소유권 확인 질문에 답하기

AI가 물어봅니다:

이 파일/코드는 본인이 직접 만들었거나, 분석 권한을 가진 것이 맞나요?
(예 / 아니오)

“예”라면 입력:

예

3단계: 책임 동의 질문에 답하기

AI가 다시 물어봅니다:

분석 결과는 방어·학습·본인 소유물 목적으로만 사용하며,
불법 활동(크랙·우회·키추출)에 절대 사용하지 않겠다는 것에 동의하나요?
(예 / 아니오)

동의하면 입력:

예

4단계: 분석 결과 읽기

분석이 완료되면 아래 형식의 보고서가 출력됩니다:

SoDam-Reverse 분석 보고서

1. 요약

이 코드는 사용자 로그인 기능을 담당합니다.
아이디와 비밀번호를 받아서 데이터베이스에서 확인하고,
맞으면 세션을 만들어서 로그인 상태로 만들어줍니다.

2. 주요 기능

- login() 함수 (src/login.js:15)
 - 로그인 전체 프로세스를 조율하는 메인 함수
- validateCredentials() 함수 (src/login.js:43)
 - 아이디/비밀번호를 데이터베이스와 대조하는 함수
- createSession() 함수 (src/login.js:78)
 - 로그인 성공 시 세션(로그인 유지 상태)을 만드는 함수

3. 발견사항

- 비밀번호가 해시(암호화) 없이 비교되는 부분이 보입니다 (login.js:51)
 - 보안 개선이 필요합니다 (bcrypt 사용 권장)

4. 불확실한 점

- 세션 만료 시간이 코드에 보이지 않습니다
→ 다른 파일에 설정이 있을 수 있습니다

5. 다음 확인사항

- `config.js` 파일에 세션 설정이 있는지 확인해 보세요
- 비밀번호 해시 적용 여부 검토가 필요합니다

5단계: 보고서 저장 위치 확인

분석 결과는 자동으로 저장됩니다:

[플러그인 폴더]/`.sodam-re/reports/`

언제든지 `/re-report`로 다시 볼 수 있습니다.

6. 보안·데이터 흐름 상세

내 코드 데이터는 어디로 가나요?

많은 분들이 “내 코드가 외부로 유출되지 않나요?” 걱정합니다. 아래 흐름으로 정확히 설명합니다:

- [1] 내 코드 (내 컴퓨터에 있음)
 - ↓ 분석을 위해 AI에 전달
- [2] Claude AI (Anthropic 서버에서 처리)
 - 코드 내용을 읽고 분석합니다
 - 분석 결과만 내 컴퓨터로 반환합니다
 - ↓ 분석 결과 반환
- [3] 내 컴퓨터 `.sodam-re/` 폴더에 저장

중요한 사실: - 코드 내용은 분석을 위해 Anthropic 서버로 전달됩니다 (Claude API 이용 약관이 적용됩니다) - 분석 결과, 동의 기록, 차단 로그는 외부로 전송되지 않습니다 - 코드 중 발견된 비밀번호·키·토큰은 `....`(마스킹됨)으로 표시 후 저장됩니다

3층 안전장치 상세 설명

1층: AI 거부 규칙 (SKILL.md)

AI 자체가 위험한 내용을 출력하지 않도록 하는 규칙입니다.

작동 방식: - 분석 요청이 들어오면 AI가 먼저 판단합니다 - “크랙 방법을 알려달라”는 요청이면 AI가 거부합니다 - 이 판단은 AI 내부에서 이루어집니다

차단 대상: - 크랙 코드 생성 - DRM 우회 방법 - 인증 우회 코드 - 비밀번호 추출 - 라이선스 우회

2층: deny-hook (re-deny-guard.mjs)

Claude Code가 실행하는 모든 도구를 실시간으로 감시하는 안전장치입니다.

작동 방식:

AI가 코드를 쓰려 함 → deny-hook 감시 → 위험 패턴 감지?
↳ 없음: 계속 진행
↳ 있음: 즉시 차단 + 사용자에게 알림

차단 패턴 (43개): crack, keygen, bypass, patch 관련 등 위험 키워드를 포함한 파일 쓰기/실행을 차단합니다.

fail-closed 원칙: hook 자체에 오류가 생기면 → “통과”가 아닌 **즉시 중단**입니다. 이것이 “fail-closed”입니다. 오류가 생겨도 위험 작업이 실행되지 않습니다.

3층: 무결성 점검 (_selftest.mjs)

안전파일 자체가 변조됐는지 SHA-256 해시로 확인합니다.

작동 방식:

/re-selftest 실행 → 핵심 파일들의 현재 해시 계산
→ 저장된 해시와 비교
↳ 일치:  정상
↳ 불일치:  변조 감지 (분석 중단)

SHA-256이란? 파일의 “디지털 지문”입니다. 파일 내용이 1글자만 바뀌어도 해시가 완전히 달라집니다. 이를 통해 안전파일이 변조됐는지 감지합니다.

마스킹 처리

코드 중에서 발견되는 민감 정보는 자동으로 마스킹됩니다:

발견 패턴	보고서 출력
password = "abc123"	password = "••••(마스킹됨)"
api_key = "sk-abc..."	api_key = "••••(마스킹됨)"
token = "Bearer xyz..."	token = "••••(마스킹됨)"

발견 패턴

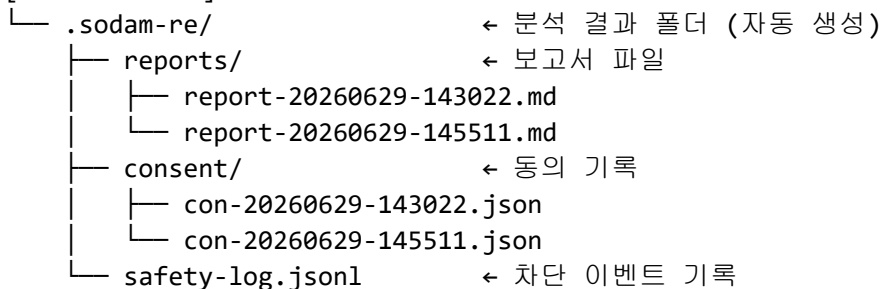
보고서 출력

```
SECRET_KEY = "super_secret" SECRET_KEY = "....(마스킹됨)"
```

10개의 마스킹 패턴이 등록되어 있습니다 (references/mask-patterns.json).

분석 결과 저장 구조

[플러그인 폴더]/

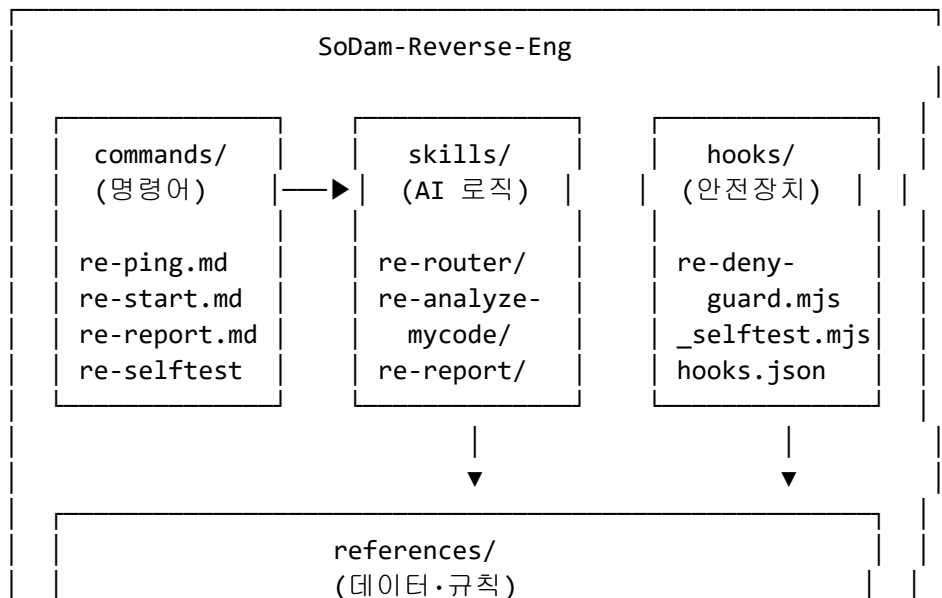


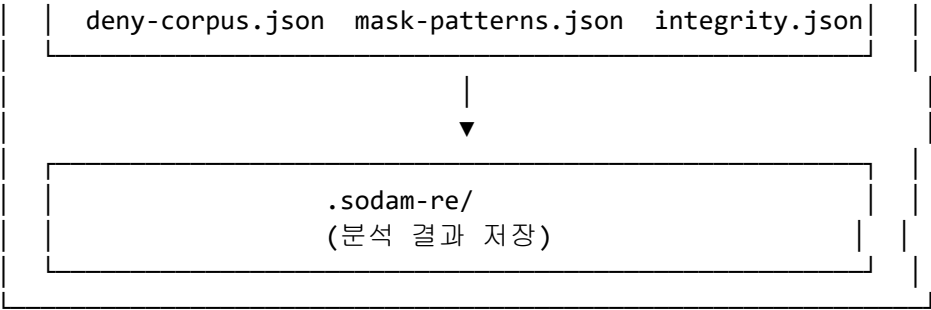
- **reports/**: 분석 보고서 (마크다운 형식)
- **consent/**: 동의 기록 (타임스탬프, 동의 여부)
- **safety-log.jsonl**: 차단 이벤트 로그 (원문은 해시 처리됨)

중요: .sodam-re/ 폴더는 .gitignore에 등록되어 있습니다. Git으로 코드를 관리해도 분석 결과가 업로드되지 않습니다.

7. 아키텍처 (내부 구조) 상세

전체 구조 다이어그램





각 구성요소 설명

commands/ — 명령어 정의

사용자가 /re-start 같은 명령어를 입력했을 때 어떻게 동작할지 정의합니다.

파일	담당 명령어	역할
re-ping.md	/re-ping	설치 확인 진단
re-start.md	/re-start	분석 시작 + 동의 게이트
re-report.md	/re-report	보고서 재출력
re-selftest.md	/re-selftest	3층 안전장치 점검

skills/ — AI 분석 로직

실제 분석이 이루어지는 AI 로직을 담고 있습니다.

폴더	역할	상태
re-router/	1층 안전규칙 + 요청 분류	✅ 활성화
re-analyze-mycode/	소스코드 분석	✅ 활성화
re-report/	보고서 생성	✅ 활성화
re-analyze-android/	Android APK 분석	⌚ Phase 2
re-analyze-binary/	실행파일 분석	⌚ Phase 3

hooks/ — 안전장치

Claude Code가 실행하는 모든 도구를 감시하고 제어합니다.

파일	역할
hooks.json	hook 설정 (어떤 도구를 감시할지)
re-deny-guard.mjs	2층: 위험 패턴 실시간 차단

파일	역할
_selftest.mjs	3층: SHA-256 무결성 점검

hooks.json의 \${CLAUDE_PLUGIN_ROOT}:

`${CLAUDE_PLUGIN_ROOT}` 는 플러그인이 설치된 폴더를 자동으로 가리킵니다. 이 덕분에 어떤 컴퓨터에서도 경로를 고치지 않아도 됩니다.

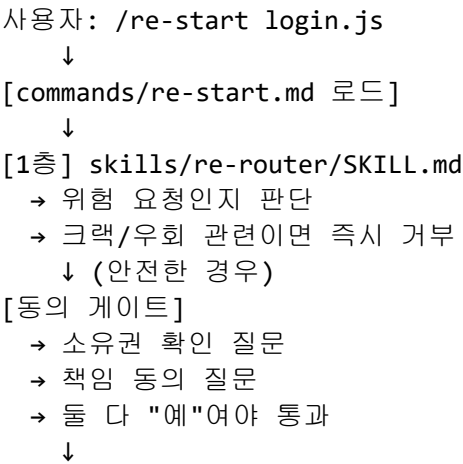
references/ — 데이터·규칙 파일

파일	내용
deny-corpus.json	차단할 위험 패턴 43개
mask-patterns.json	마스킹할 민감 정보 패턴 10개
trust-catalog.md	신뢰하는 외부 도구 목록 15개
report-template.md	보고서 표준 양식
integrity.json	안전파일 SHA-256 해시 저장소

scripts/ — 유틸리티

파일	기능
re-inject-harness.mjs	Harness와 규칙 공유
re-inject-context.mjs	Context와 스코프 연결
check-family.mjs	6형제 상태 전체 확인
check-trust-freshness.mjs	신뢰 도구 신선도 점검

요청 처리 흐름 (기술적 상세)



- [2층] hooks/re-deny-guard.mjs (병렬 감시)
 - 모든 도구 호출을 실시간 감시
 - 위험 패턴 감지 시 즉시 차단
- ↓
- [분석 실행] skills/re-analyze-mycode/SKILL.md
 - Read 도구로 파일 읽기 (실행하지 않음)
 - 경로 검증 (.../, 심볼릭 링크 차단)
 - 마스킹 처리
- ↓
- [보고서 생성] skills/re-report/SKILL.md
 - 표준 보고서 양식으로 정리
 - 한국어 설명 추가
- ↓
- [저장] .sodam-re/reports/에 저장
- ↓
- [3층] 주기적 무결성 점검 (_selftest.mjs)
 - 안전파일 변조 여부 확인

8. 파일·문서 위치 가이드

사용자가 자주 찾는 파일

찾는 것	파일 위치
이 가이드	GUIDE.md (현재 파일)
빠른 개요	README.md
영어 개요	README.en.md
영어 가이드	GUIDE.en.md
오류 해결	TROUBLESHOOTING.md
분석 결과	.sodam-re/reports/
동의 기록	.sodam-re/consent/
차단 로그	.sodam-re/safety-log.jsonl
라이선스 전문	LICENSE
저작권 고지	NOTICE

개발자가 자주 찾는 파일

찾는 것	파일 위치
명령어 정의	commands/re-start.md 등
AI 분석 로직	skills/re-analyze-mycode/SKILL.md

찾는 것	파일 위치
보고서 로직	skills/re-report/SKILL.md
안전 규칙	skills/re-router/SKILL.md
차단 패턴 데이터	references/deny-corpus.json
마스킹 패턴	references/mask-patterns.json
보고서 양식	references/report-template.md
hook 설정	hooks/hooks.json
위험 차단 코드	hooks/re-deny-guard.mjs
무결성 점검 코드	hooks/_selftest.mjs
해시 저장소	references/integrity.json
개발 진행 상태	CHECKPOINT.md

전체 폴더 트리

SoDam-Reverse-Eng/	← 플러그인 루트 폴더
├── .claude-plugin/	← 플러그인 선언
│ ├── plugin.json	
│ └── marketplace.json	
├── commands/	← 명령어 4개 (+ 예정 2개)
│ ├── re-ping.md	
│ ├── re-start.md	
│ ├── re-report.md	
│ ├── re-selftest.md	
│ ├── re-android.md	← Phase 2 예정
│ └── re-binary.md	← Phase 3 예정
├── skills/	← AI 로직 5개
│ ├── re-router/SKILL.md	← 1층 안전규칙
│ ├── re-analyze-mycode/SKILL.md	← 소스코드 분석
│ ├── re-report/SKILL.md	← 보고서 생성
│ ├── re-analyze-android/	← Phase 2 예정
│ └── re-analyze-binary/	← Phase 3 예정
├── hooks/	← 안전장치
│ ├── hooks.json	
│ ├── re-deny-guard.mjs	← 2층: 위험 차단
│ └── _selftest.mjs	← 3층: 무결성 점검
├── references/	← 데이터·규칙
│ ├── deny-corpus.json	
│ ├── mask-patterns.json	
│ ├── trust-catalog.md	
│ └── report-template.md	

└─ integrity.json	
└─ scripts/	← 유틸리티
└─ re-inject-harness.mjs	
└─ re-inject-context.mjs	
└─ check-family.mjs	
└─ check-trust-freshness.mjs	
└─ samples/	← 테스트용 예제
└─ safe-login.js	
└─ deny-demo.txt	
└─ .sodam-re/	← 분석 결과 (자동 생성)
└─ README.md / README.en.md	
└─ GUIDE.md / GUIDE.en.md	
└─ TROUBLESHOOTING.md	
└─ CHECKPOINT.md	
└─ SETUP_BLOCKED_FILES.md	
└─ LICENSE	
└─ NOTICE	

9. 고급 기능 안내

형제 시너지 스크립트

re-inject-harness.mjs

```
node scripts/re-inject-harness.mjs
```

목적: Harness의 safety-rules에 RE 위험 패턴을 등록 **효과:** hook 1개가 Harness + Reverse 규칙 모두 담당 (중복 없음) **실행 시기:** Harness 설치 후 최초 1회

기대 출력:

RE 규칙 8개 (catastrophic) 주입 완료
 RE 규칙 10개 (risky) 주입 완료
 저장: ~/.sodamharness/safety-rules.json

re-inject-context.mjs

```
node scripts/re-inject-context.mjs
```

목적: Context가 RE 스코프 누락을 감지하도록 검진 항목 추가 **효과:** CLAUDE.md에 “방어·교육·본인소유물 전용” 언급 없으면 경고 **실행 시기:** Context 설치 후 최초 1회

check-family.mjs

```
node scripts/check-family.mjs
```

목적: 6형제 전체 설치 상태 + 시너지 활성화 현황 확인 **실행 시기:** 언제든지 (문제 진단 시 유용)

check-trust-freshness.mjs

```
node scripts/check-trust-freshness.mjs
```

목적: 신뢰 카탈로그(trust-catalog.md)의 도구들이 최신 버전인지 확인 **실행 시기:** 분기별 1회 권장

무결성 해시 수동 갱신

안전파일을 합법적으로 수정했다면 해시를 새로 등록해야 합니다:

```
node hooks/_selftest.mjs
```

출력에서 새 SHA-256 해시 복사 → references/integrity.json 업데이트

테스트용 예제 파일

정상 분석 테스트:

```
/re-start samples/safe-login.js
```

→ 크랙·우회가 없는 일반 코드. 분석이 정상 진행돼야 합니다.

차단 테스트:

```
/re-start samples/deny-demo.txt
```

→ 위험 키워드가 포함된 파일. 분석이 차단돼야 합니다.

10. 문제 해결·오류 대처

더 많은 오류 해결: [TROUBLESHOOTING.md](#)

Q1. 명령어가 안 뜨거나 없어요

증상: /re-start나 /re-ping을 입력해도 아무 반응이 없거나 자동완성 목록에 나타나지 않는다.

원인 (99%가 이것): Claude Code를 홈 폴더(c:\Users\이름)에서 시작했거나, 채팅창에서 cd 명령어로 폴더를 이동했습니다.

해결: 1. 채팅 창에서 /exit 입력 → Enter 2. PowerShell 창 **완전히 닫기** 3. PowerShell **새로 열기** 4. **프로젝트 폴더에서** Claude Code 시작: powershell cd D:\내프로젝트폴더 claude 5. /re-ping 입력 → "Pong!" 응답 확인

Q2. /re-ping 은 되는데 /re-start 가 안 돼요

증상: /re-ping은 "Pong!" 응답이 오는데, /re-start 파일경로를 입력하면 분석이 안 된다.

원인 A — 파일 경로가 틀렸다:

/re-start 없는파일.js ← 파일이 없으면 분석 불가

해결: 파일이 실제로 존재하는지 확인.

원인 B — 동의에서 “아니오” 했다: 소유권 또는 책임 동의 질문에 “아니오”나 애매한 답변을 했으면 분석이 중단됩니다.

해결: /re-start 파일경로를 다시 입력하고, 두 질문에 모두 “예” 입력.

Q3. “Node.js를 찾을 수 없습니다” 오류

원인: Node.js가 설치 안 됐거나, 설치 후 재부팅을 안 했다.

해결: 1. nodejs.org → LTS 버전 설치 2. 컴퓨터 재부팅 3. PowerShell 새로 열기 4. node --version 입력 → v18 이상 확인

Q4. 정상 작업이 막혀요 (Harness·Loop 설치 시)

원인: Harness 또는 Loop가 설치된 상태에서 홈 폴더(c:\Users\이름)에서 Claude Code를 시작했다.

해결:

```
cd D:\내프로젝트폴더
claude
```

프로젝트 폴더에서 Claude Code를 다시 시작하세요.

Q5. 동의 질문에서 계속 막혀요

증상: “예”라고 했는데 AI가 다시 물어본다.

원인: 동의 표현이 너무 애매하다.

해결: 아래 중 하나로 명확하게 입력하세요:

```
예
네
동의합니다
yes
```

“그런 것 같아요”, “아마도요”, “당연하죠” 같은 표현은 동의로 처리되지 않습니다.

Q6. /re-selftest에서 일부 항목이 ✖

1층 ✖인 경우:

skills/re-router/SKILL.md 파일 확인:

```
ls D:\내플러그인폴더\skills\re-router\SKILL.md
```

파일이 없으면 → SETUP_BLOCKED_FILES.md에서 내용 복사 후 파일 생성

2층 ✖인 경우:

hooks/re-deny-guard.mjs 파일 확인:

```
ls D:\내플러그인폴더\hooks\re-deny-guard.mjs
```

파일이 없으면 → SETUP_BLOCKED_FILES.md에서 내용 복사 후 파일 생성

3층 ✖인 경우 (해시 불일치):

```
node D:\내플러그인폴더\hooks\_selftest.mjs
```

출력된 해시 복사 → references/integrity.json 업데이트 → 재실행

Q7. 보고서에서 비밀번호가 가 아니라 그대로 보여요

심각도: 높음 (즉시 처리 필요)

해결: 1. 해당 보고서 파일 삭제 2. 이메일(startmxk@gmail.com) 또는 [GitHub Issues](#)에 신고

보고서 파일 삭제:

`Remove-Item D:\내플러그인폴더\.sodam-re\reports\report-문제파일.md`

Q8. 분석이 너무 오래 걸려요

해결: 파일 하나씩 분석하세요:

```
/re-start src/auth.js
```

폴더 전체보다 파일 단위가 훨씬 빠릅니다.

Q9. “분석 대상 경로가 허용 범위를 벗어납니다” 오류

해결:

```
/re-start 상대경로/파일.js ← 현재 폴더 기준 상대 경로 사용
```

.. 포함 경로나 시스템 폴더는 보안상 차단됩니다.

Q10. Harness 시너지가 안 연결돼요

```
node scripts/re-inject-harness.mjs
```

“RE 규칙 N개 주입 완료” 메시지가 나오면 성공.

11. FAQ 자주 묻는 질문 25가지

Q1. 내 코드를 AI에게 보내면 유출되지 않나요?

코드 내용은 분석을 위해 Anthropic 서버로 전달됩니다. 이는 Claude API의 정상적인 작동 방식이며 Anthropic 이용 약관이 적용됩니다. 분석 결과는 내 컴퓨터에만 저장됩니다.

Q2. 이 도구를 쓰면 내 코드가 AI 학습에 쓰이나요?

이는 Anthropic의 API 이용 약관에 따릅니다. Anthropic 공식 사이트의 개인정보 처리 방침을 확인하세요.

Q3. 크랙 요청을 했는데 거부됐어요. 왜요?

의도적으로 거부하도록 설계했습니다. 크랙, DRM 우회, 인증 우회, 비밀번호 추출은 이 도구의 기능 범위 밖입니다.

Q4. 상업적으로 사용해도 되나요?

네. Apache License 2.0에 따라 상업적 사용이 허용됩니다. 단, LICENSE 파일과 저작권 고지를 포함해야 합니다. 분석 대상 소프트웨어의 라이선스는 별도로 확인하세요.

Q5. 무료인가요?

이 플러그인 자체는 무료(Apache-2.0)입니다. Claude API 사용료는 Anthropic에서 별도로 청구됩니다.

Q6. 여러 파일을 한 번에 분석할 수 있나요?

현재는 파일 하나씩 분석합니다. 여러 파일은 순서대로 분석하세요.

Q7. 분석 중 발견한 비밀번호는 어디에 저장되나요?

....(마스킹됨)으로 처리된 형태로 보고서에 포함됩니다. 원문 비밀번호는 저장되지 않습니다.

Q8. 분석 결과를 삭제하고 싶어요

.sodam-re/reports/ 폴더의 보고서 파일을 직접 삭제하면 됩니다.

Q9. 분석 결과를 다른 사람과 공유해도 되나요?

마스킹 처리가 제대로 됐는지 확인 후 공유하세요. 보고서에 민감 정보가 없는지 검토 필요합니다.

Q10. Windows가 아닌 Mac·Linux에서도 쓸 수 있나요?

Phase 1은 Windows 기준으로 개발됐습니다. Mac·Linux 지원은 Phase 2·3에서 추가 될 예정입니다.

Q11. 분석 후 보고서가 어디 저장됐는지 모르겠어요

/re-report 입력 시 화면에 출력됩니다. 또는 .sodam-re/reports/ 폴더를 직접 확인하세요.

Q12. 이 도구가 코드를 실행하나요?

절대 실행하지 않습니다. 읽기 전용입니다.

Q13. 어떤 프로그래밍 언어를 지원하나요?

JavaScript, Python, Java, C/C++, Go, Rust, PHP, TypeScript 등 주요 언어를 지원합니다. AI가 이해하는 언어라면 대부분 분석 가능합니다.

Q14. APK 파일(Android 앱)을 분석할 수 있나요?

Phase 2에서 지원 예정입니다. 현재는 소스코드 파일만 가능합니다.

Q15. exe 파일(실행파일)을 분석할 수 있나요?

Phase 3에서 지원 예정입니다. 현재는 소스코드 파일만 가능합니다.

Q16. 보고서에 틀린 내용이 있어요

AI는 100% 정확하지 않을 수 있습니다. 보고서의 “불확실한 점” 섹션을 확인하고, 중요한 결정 전에는 직접 코드를 검토하세요.

Q17. 이 도구를 수정해서 쓸 수 있나요?

Apache-2.0 라이선스에 따라 수정 가능합니다. 수정한 사실을 명시하고, LICENSE 파일을 포함해야 합니다.

Q18. 새 기능을 제안하고 싶어요

이메일(startmxk@gmail.com)로 제안해 주세요. 저장소 초대권을 받은 분은 [GitHub Issues](#)에도 작성 가능합니다.

Q19. 버그를 발견했어요

이메일(startmxk@gmail.com)로 신고해 주세요. 어떤 명령어를 입력했는지, 어떤 결과가 나왔는지, 기대했던 결과가 무엇인지 포함해 주세요. 저장소 초대를 받은 분은 [GitHub Issues](#)에도 신고 가능합니다.

Q20. 인터넷 연결이 없어도 작동하나요?

아니요. Claude AI가 Anthropic 서버에서 동작하므로 인터넷 연결이 필요합니다.

Q21. 이 플러그인을 지우고 싶어요

```
/plugin uninstall sodam-reverse@sodamreverse-marketplace
```

또는 플러그인 폴더 전체를 삭제하고 Claude Code 재시작.

Q22. 여러 프로젝트에서 동시에 쓸 수 있나요?

네. 플러그인은 한 번 설치하면 다른 프로젝트에서도 사용 가능합니다. 단, 각 프로젝트 폴더에서 Claude Code를 시작해야 합니다.

Q23. 회사(법인)에서 사용해도 되나요?

Apache-2.0에 따라 법인도 상업적 사용이 가능합니다. 단, 분석 대상 소프트웨어의 라이선스와 Claude API 이용 약관을 별도로 확인하세요.

Q24. 안전장치를 끌 수 있나요?

아니요. 안전 3층은 끌 수 없도록 설계됐습니다. 이것은 의도적인 설계입니다.

Q25. Harness 없이도 쓸 수 있나요?

네. Harness는 선택 사항입니다. 없어도 RE 자체 hook(2층)이 작동합니다.

12. 라이선스·저작권·상업적 용도 상세

적용 라이선스

Apache License, Version 2.0
Copyright (c) 2026 SoDam AI Studio

Apache 2.0은 오픈소스 라이선스 중 가장 기업 친화적인 라이선스 중 하나입니다. MIT 보다 특허 조항이 명확하며, GPL보다 제약이 적습니다.

허용 사항 (명시적으로 허용)

행위	허용 여부	조건
개인적 사용	 허용	없음
수정 · 변경	 허용	변경 사실 명시
복제 · 포크	 허용	라이선스 · 저작권 고지 포함
재배포 (무료)	 허용	라이선스 · 저작권 고지 포함
재배포 (유료)	 허용	라이선스 · 저작권 고지 포함
상업적 사용	 허용	라이선스 · 저작권 고지 포함
특허 사용	 허용	특허 소송 시 자동 해지
사적 수정	 허용	배포 안 하면 조건 없음

의무 사항 (반드시 해야 하는 것)

1. LICENSE 파일 포함

소스코드나 바이너리를 배포할 때 반드시 LICENSE 파일을 포함해야 합니다.

2. 저작권 고지 보존

Copyright (c) 2026 SoDam AI Studio

이 문구를 제거하거나 변경하면 안 됩니다.

3. 변경 사실 명시

원본 코드를 수정했다면, 어떤 파일을 언제 수정했는지 명시해야 합니다.

4. NOTICE 파일 동봉

NOTICE 파일이 있다면 배포 시 함께 포함해야 합니다.

금지 사항 (하면 안 되는 것)

행위	설명
“SoDam” 상표 무단 사용	허가 없이 자신의 제품·서비스 상표로 사용 불가
보증 요구	AS-IS 제공 — 품질·안전성 보증 없음
손해배상 청구	면책 조항 적용

상업적 사용 시 체크리스트

- 이 플러그인에 대해:** - ☐ LICENSE 파일 포함됨 - ☐ 저작권 고지 © 2026 SoDam AI Studio 보존됨 - ☐ 수정 사항 있으면 변경 이력 기록됨 - ☐ NOTICE 파일 포함됨
- 분석 대상 소프트웨어에 대해:** - ☐ 분석 대상의 EULA·라이선스를 별도 확인함 - ☐ 분석 허가를 받았거나 본인 소유임을 확인함
- Claude API 사용에 대해:** - ☐ Anthropic 이용 약관 확인함 - ☐ 상업적 API 사용 조건 충족함

타사 상표·저작권 고지

소프트웨어	소유자	공식 제휴 여부
Claude, Claude Code	Anthropic PBC	아님
IDA Pro (Phase 3 선택)	Hex-Rays SA	아님
Node.js	OpenJS Foundation	아님

위 상표들은 각 소유자의 재산이며, 이 플러그인은 공식 제휴·보증 관계가 없습니다.

면책 조항

이 소프트웨어는 교육·방어·연구 목적으로 제공됩니다.

사용자는 다음에 동의한 것으로 간주됩니다: - 분석 대상 코드의 합법적 소유자이거나 허가받은 자임 - 분석 결과를 합법적 목적으로만 사용함 - 이 도구 사용으로 인한 모든 결과에 책임을 짐

제작자(SoDam AI Studio)는 다음에 책임지지 않습니다: - 이 도구의 오분석으로 인한 손실 - 이 도구를 불법 목적으로 사용한 결과 - Claude API 서비스 중단으로 인한 불편

라이선스 전문: [LICENSE](#) · 저작권 고지: [NOTICE](#)

한국어 가이드 / English version: [GUIDE.en.md](#) 빠른 개요: [README.md](#) 오류 해결: [TROUBLESHOOTING.md](#)